WEB X.0– EXPERIMENT NO. 04

Advanced TypeScript using OOP & Data Persistence

Aim:

To implement advanced TypeScript concepts using:

Classes , Objects , OOP principles

LAB OUTCOME

After this experiment, students will be able to:

- Create **classes in TypeScript**
- Use **OOP concepts**
- Understand **how data is stored**
- Write **industry-style backend code**

THEORY

What is TypeScript?

TypeScript is a **typed version of JavaScript**.

- JavaScript allows mistakes
- TypeScript **prevents mistakes before running the program**

Example: `letage:number =20;`

What is OOP?

OOP means **Object-Oriented Programming**.

OOP is based on: **Classes** , **Objects** , **Data + Functions together**

PART 1: BANK CLASS

What Is a Class?

A **class is a blueprint**.

It decides: What data we store , What actions we perform

Bank Class

A bank account needs: Account number , Balance , Deposit , Withdraw , Check balance

Constructor : Constructor runs **automatically** when object is created

It **initializes values**

Deposit Method

- Deposit only if amount is positive
- Balance increases

Withdraw Method

- Prevents negative balance
- Protects account

Get Balance

Balance can be **read**, not modified directly

```
class Bank {
  public accountNumber: string;
  private balance: number;
  constructor(accountNumber: string, initialBalance: number) {
    this.accountNumber = accountNumber;
    this.balance = initialBalance;
  }
  deposit(amount: number): void {
    if (amount > 0) {
      this.balance += amount;
      console.log(`Deposited ₹${amount}`);
    }
  }
  withdraw(amount: number): void {
    if (amount > 0 && this.balance >= amount) {
      this.balance -= amount;
      console.log(`Withdrawn ₹${amount}`);
    } else {
      console.log("Insufficient balance");
    }
  }
  getBalance(): number {
    return this.balance;
  }
}
const bankAccount = new Bank("1234567890", 1000);
bankAccount.deposit(500);
bankAccount.withdraw(300);
console.log("Current Balance:", bankAccount.getBalance());
```

Object Creation (Very Important)

- `new` creates an object
- Constructor is called

- Memory is allocated

```
PS C:\Users\Dell\Desktop\231441> node bank.ts
PS C:\Users\Dell\Desktop\231441> node bank.ts
Deposited ₹500
Withdrawn ₹300
Current Balance: 1200
PS C:\Users\Dell\Desktop\231441>
```

PART 2: DATA PERSISTENCE (DATABASE CONNECTIVITY)

What Is Data Persistence?

Data persistence means:

Data is not lost after program stops

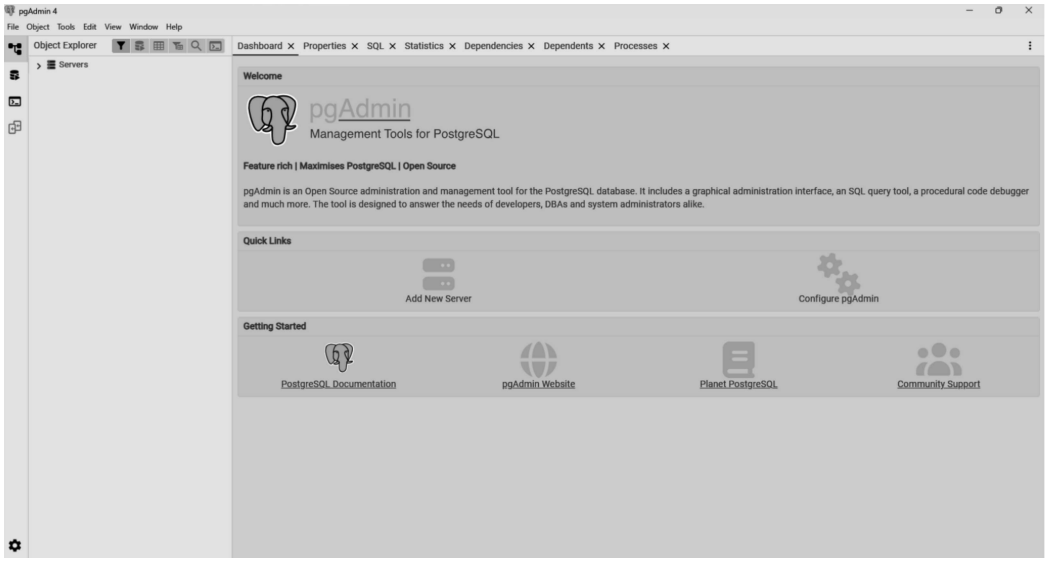
Example:

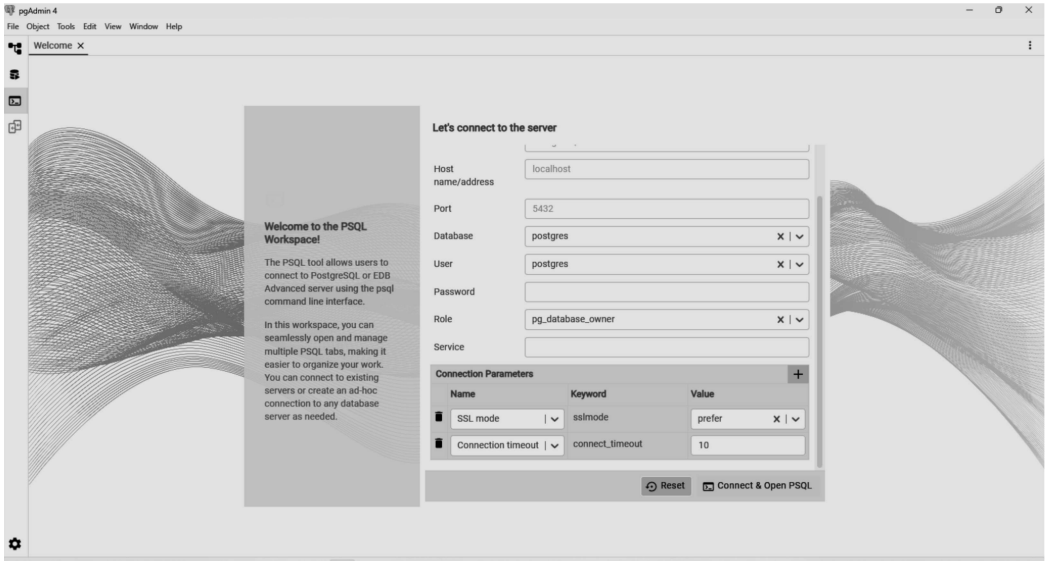
- \ATM balance
- Student records
- Login details+

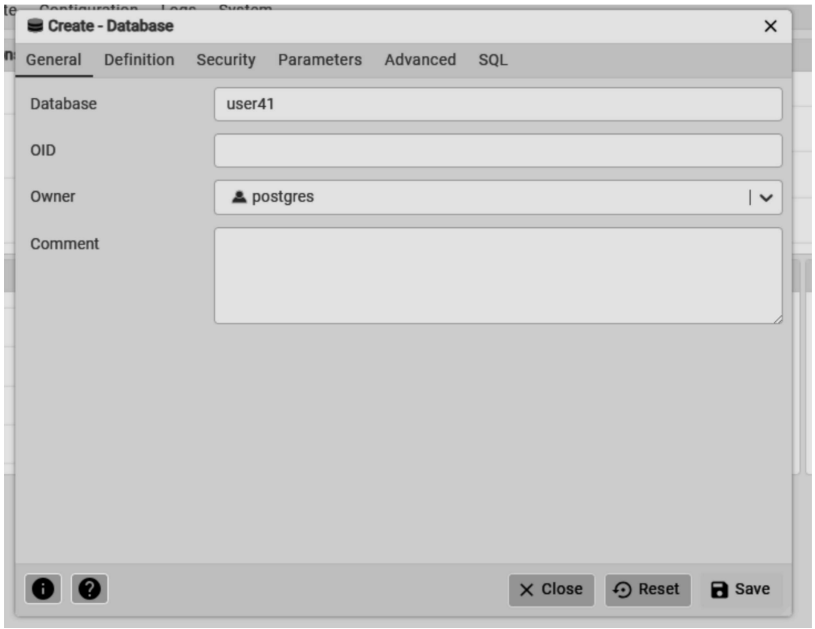
Types of Data Storage (Explain Simply)

Method	Meaning
Local Storage	Browser memory
IndexedDB	Advanced browser DB
Server Database	Real-world apps

In this experiment, we use **Server Database (PostgreSQL)**


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers
Dashboard Properties SQL Statistics Dependencies Dependents Processes
Welcome
pgAdmin
Management Tools for PostgreSQL
Feature rich | Maximises PostgreSQL | Open Source
pgAdmin is an Open Source administration and management tool for the PostgreSQL database. It includes a graphical administration interface, an SQL query tool, a procedural code debugger and much more. The tool is designed to answer the needs of developers, DBAs and system administrators alike.
Quick Links
Add New Server
Configure pgAdmin
Getting Started
PostgreSQL Documentation
pgAdmin Website
Planet PostgreSQL
Community Support


pgAdmin 4
File Object Tools Edit View Window Help
Welcome
Welcome to the PSQL Workspace!
The PSQL tool allows users to connect to PostgreSQL or EDB Advanced server using the psql command line interface.
In this workspace, you can seamlessly open and manage multiple PSQL tabs, making it easier to organize your work. You can connect to existing servers or create an ad-hoc connection to any database server as needed.
Let's connect to the server
Host name/address
localhost
Port
5432
Database
postgres
User
postgres
Password
Role
pg_database_owner
Service
Connection Parameters
Name
Keyword
Value
SSL mode
sslmode
prefer
Connection timeout
connect_timeout
10
Reset
Connect & Open PSQL


Create - Database
General
Definition
Security
Parameters
Advanced
SQL
Database
user41
OID
Owner
postgres
Comment
Close
Reset
Save

SETUP STEPS

Step 1: Create Project

```
npm init -y
```

```
Wrote to C:\Users\Dell\Desktop\package.json:
C:\Users\Dell\Desktop>mkdir 231441-webx-exp-04
{
  "name": "desktop",p>npm init -y
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "devDependencies": {
    "@types/node": "^25.2.0",
    "ts-node": "^10.9.2",
    "typescript": "^5.9.3"
  },
  "dependencies": {
    "pg": "^8.18.0",
    "typeorm": "^0.3.28"
  }
}



C:\Users\Dell\Desktop>npm install

up to date, audited 139 packages in 1s

36 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
PS C:\Users\Dell\Desktop\231441-webx-exp-04> npm install

up to date, audited 139 packages in 936ms

36 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Step 2: Install TypeScript

```
npm install typescript ts-node @types/node --save-dev
```

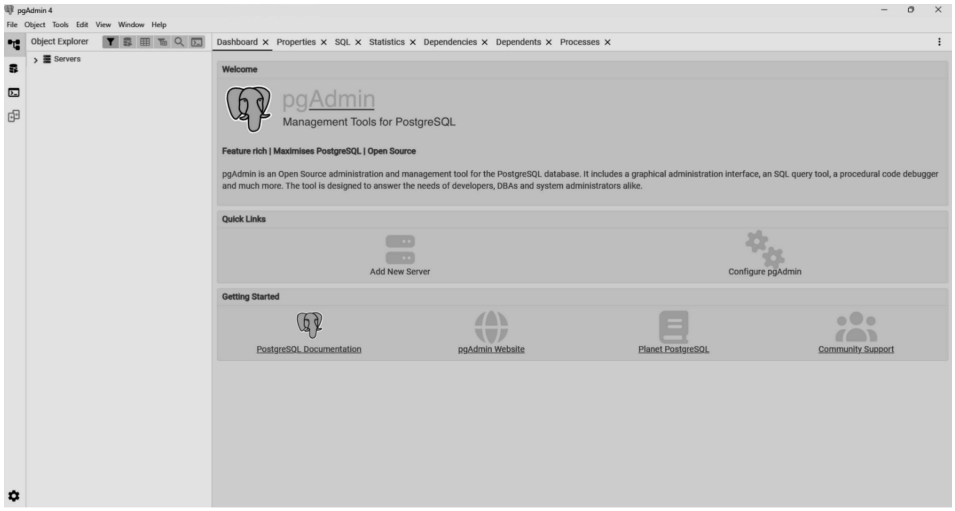
```
PS C:\Users\Dell\Desktop\231441-webx-exp-04> npm install typescript ts-node @types/node --save-dev
added 20 packages, and audited 21 packages in 6s

found 0 vulnerabilities
PS C:\Users\Dell\Desktop\231441-webx-exp-04> npm install typeorm pg
ons of glob are not supported, and contain widely publicized security vulnerabilities, which have bee
for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 118 packages, and audited 139 packages in 33s

36 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

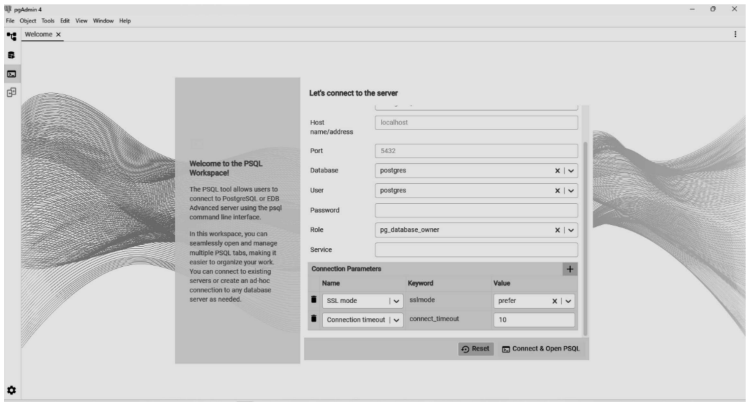
```
PS C:\Users\Dell\Desktop\231441-webx-exp-04> npx tsc --init

 Created a new tsconfig.json

 You can learn more at https://aka.ms/tsconfig
```

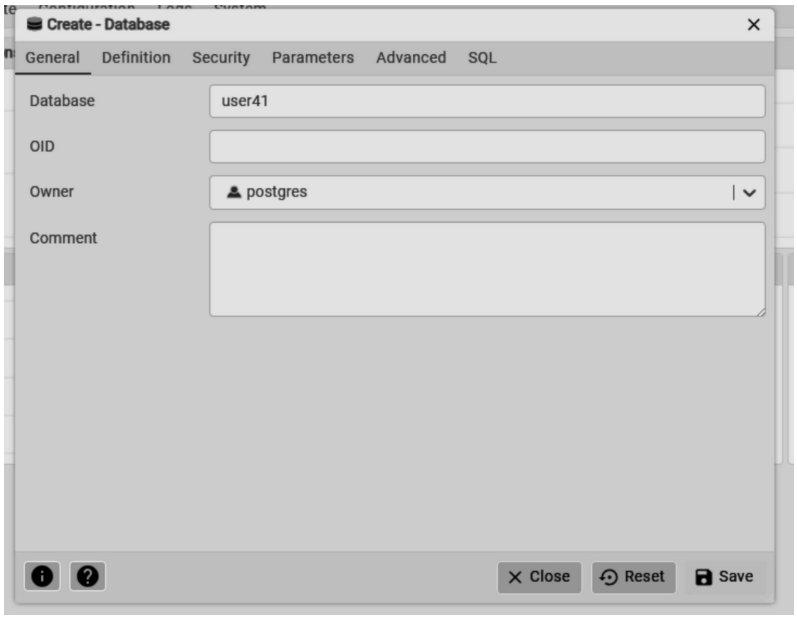
```
PS C:\Users\Dell\Desktop\231441-webx-exp-04> npm init -y
Wrote to C:\Users\Dell\Desktop\231441-webx-exp-04\package.json:
{
  "name": "231441-webx-exp-04",
  "version": "1.0.0",
  "description": "",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Step 3: Install Database Tools

```
npm install typeorm pg
```


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers
Dashboard Properties SQL Statistics Dependencies Dependents Processes
Welcome
pgAdmin
Management Tools for PostgreSQL
Feature rich | Maximises PostgreSQL | Open Source
pgAdmin is an Open Source administration and management tool for the PostgreSQL database. It includes a graphical administration interface, an SQL query tool, a procedural code debugger and much more. The tool is designed to answer the needs of developers, DBAs and system administrators alike.
Quick Links
Add New Server
Configure pgAdmin
Getting Started
PostgreSQL Documentation
pgAdmin Website
Planet PostgreSQL
Community Support


pgAdmin 4
File Object Tools Edit View Window Help
Welcome
Welcome to the PSQL Workspace!
The PSQL tool allows users to connect to PostgreSQL or EDB Advanced server using the psql command line interface.
In this workspace, you can seamlessly open and manage multiple PSQL tabs, making it easier to organize your work. You can connect to existing servers or create an ad-hoc connection to any database server as needed.
Let's connect to the server
Host name/address
localhost
Port
5432
Database
postgres
User
postgres
Password
Role
pg_database_owner
Service
Connection Parameters
Name
Keyword
Value
SSL mode
sslmode
prefer
Connection timeout
connect_timeout
10
Reset
Connect & Open PSQL


Create - Database
General
Definition
Security
Parameters
Advanced
SQL
Database
user41
OID
Owner
postgres
Comment
Close
Reset
Save

ENTITY CREATION (Database Table)

What Is an Entity?

An **entity is a table** in database.

User Entity

User.ts

```
import { Entity, PrimaryGeneratedColumn, Column } from "typeorm";

@Entity()
export class User {

  @PrimaryGeneratedColumn()
  id!: number;

  @Column()
  name!: string;

  @Column()
  email!: string;

  @Column()
  password!: string;
}
```

DATABASE CONNECTION

index.ts

```
import "reflect-metadata";

import { DataSource } from "typeorm";
import { User } from "./entity/User";
const AppDataSource = new DataSource({
  type: "postgres",
  host: "localhost",
  port: 5432,
  username: "postgres",
  password: "1234",
  database: "user41",
  entities: [User],
  synchronize: true,
});
async function main() {
  await AppDataSource.initialize();
  console.log("Database Connected Successfully!");
  const userRepo = AppDataSource.getRepository(User);
  // INSERT DATA
  const newUser = userRepo.create({
```

```
    name: "Raavi Yashfeen Nadim",
    email: "yashfeen.231441.it@mhssce.ac.in",
    password: "12345",
  });
  await userRepo.save(newUser);
  console.log("User Saved:", newUser);
  // FETCH DATA
  const users = await userRepo.find();
  console.log("All Users:", users);
}
main();
```

Tsconfig.json

```
  "compilerOptions": {
    "module": "commonjs",
    "target": "es2020",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

Output :

```
PS C:\Users\DBL-24\Downloads\231441> npx ts-node src/index.ts
Database Connected Successfully!
User Saved: User {
  name: 'Raavi Yashfeen Nadim',
  email: 'yashfeen.231441.it@mhssce.ac.in',
  password: '12345',
  id: 2
}
All Users: [
  User {
    id: 1,
    name: 'Raavi Yashfeen Nadim',
    email: 'yashfeen.231441.it@mhssce.ac.in',
    password: '12345'
  },
  User {
    id: 2,
    name: 'Raavi Yashfeen Nadim',
    email: 'yashfeen.231441.it@mhssce.ac.in',
    password: '12345'
  }
]
PS C:\Users\DBL-24\Downloads\231441>
```

CONCLUSION

In this experiment, we learned how TypeScript uses OOP to create secure and structured programs, and how data persistence helps store information permanently.

These concepts are used in **real banking, e-commerce, and enterprise systems**.